

B.4 Multithreaded Multiprocessor Architecture

To address different market segments, GPUs implement scalable numbers of multiprocessors—in fact, GPUs are multiprocessors composed of multiprocessors. Furthermore, each multiprocessor is highly multithreaded to execute many fine-grained vertex and pixel shader threads efficiently. A quality basic GPU has two to four multiprocessors, while a gaming enthusiast's GPU or computing platform has dozens of them. This section looks at the architecture of one such multithreaded multiprocessor, a simplified version of the NVIDIA Tesla *streaming multiprocessor* (SM) described in Section B.7.

Why use a multiprocessor, rather than several independent processors? The parallelism within each multiprocessor provides localized high performance and supports extensive multithreading for the fine-grained parallel programming models described in Section B.3. The individual threads of a thread block execute together within a multiprocessor to share data. The multithreaded multiprocessor design we describe here has eight scalar processor cores in a tightly coupled architecture, and executes up to 512 threads (the SM described in Section B.7 executes up to 768 threads). For area and power efficiency, the multiprocessor shares large complex units among the eight processor cores, including the instruction cache, the multithreaded instruction unit, and the shared memory RAM.

Massive Multithreading

GPU processors are highly multithreaded to achieve several goals:

- Cover the latency of memory loads and texture fetches from DRAM
- Support fine-grained parallel graphics shader programming models
- Support fine-grained parallel computing programming models
- Virtualize the physical processors as threads and thread blocks to provide transparent scalability
- Simplify the parallel programming model to writing a serial program for one thread

Memory and texture fetch latency can require hundreds of processor clocks, because GPUs typically have small streaming caches rather than large working-set caches like CPUs. A fetch request generally requires a full DRAM access latency plus interconnect and buffering latency. Multithreading helps cover the latency with useful computing—while one thread is waiting for a load or texture fetch to complete, the processor can execute another thread. The fine-grained parallel programming models provide literally thousands of independent threads that can keep many processors busy despite the long memory latency seen by individual threads.

A graphics vertex or pixel shader program is a program for a single thread that processes a vertex or a pixel. Similarly, a CUDA program is a C program for a single thread that computes a result. Graphics and computing programs instantiate many parallel threads to render complex images and compute large result arrays. To dynamically balance shifting vertex and pixel shader thread workloads, each multiprocessor concurrently executes multiple different thread programs and different types of shader programs.

To support the independent vertex, primitive, and pixel programming model of graphics shading languages and the single-thread programming model of CUDA C/C++, each GPU thread has its own private registers, private per-thread memory, program counter, and thread execution state, and can execute an independent code path. To efficiently execute hundreds of concurrent lightweight threads, the GPU multiprocessor is hardware multithreaded—it manages and executes hundreds of concurrent threads in hardware without scheduling overhead. Concurrent threads within thread blocks can synchronize at a barrier with a single instruction. Lightweight thread creation, zero-overhead thread scheduling, and fast barrier synchronization efficiently support very fine-grained parallelism.

Multiprocessor Architecture

A unified graphics and computing multiprocessor executes vertex, geometry, and pixel fragment shader programs, and parallel computing programs. As [Figure B.4.1](#) shows, the example multiprocessor consists of eight *scalar processor* (SP) cores each with a large multithreaded *register file* (RF), two *special function units* (SFUs), a multithreaded instruction unit, an instruction cache, a read-only constant cache, and a shared memory.

The 16 KB shared memory holds graphics data buffers and shared computing data. CUDA variables declared as `__shared__` reside in the shared memory. To map the logical graphics pipeline workload through the multiprocessor multiple times, as shown in Section B.2, vertex, geometry, and pixel threads have independent input and output buffers, and workloads arrive and depart independently of thread execution.

Each SP core contains scalar integer and floating-point arithmetic units that execute most instructions. The SP is hardware multithreaded, supporting up to 64 threads. Each pipelined SP core executes one scalar instruction per thread per clock, which ranges from 1.2 GHz to 1.6 GHz in different GPU products. Each SP core has a large RF of 1024 general-purpose 32-bit registers, partitioned among its assigned threads. Programs declare their register demand, typically 16 to 64 scalar 32-bit registers per thread. The SP can concurrently run many threads that use a few registers or fewer threads that use more registers. The compiler optimizes register allocation to balance the cost of spilling registers versus the cost of fewer threads. Pixel shader programs often use 16 or fewer registers, enabling each SP to run up to 64 pixel shader threads to cover long-latency texture fetches. Compiled CUDA programs often need 32 registers per thread, limiting each SP to 32 threads, which limits such a kernel program to 256 threads per thread block on this example multiprocessor, rather than its maximum of 512 threads.

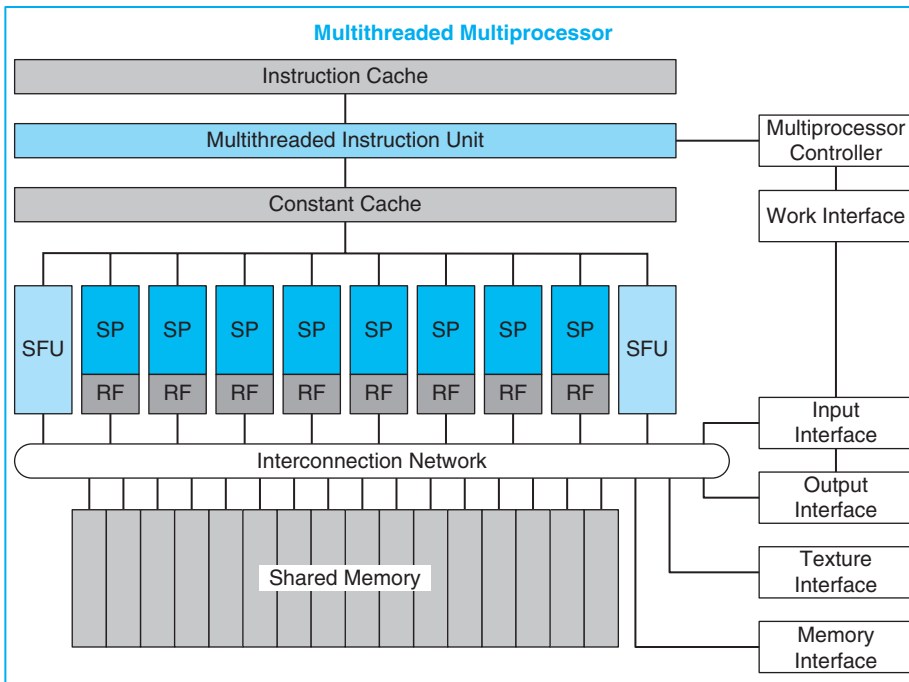


FIGURE B.4.1 Multithreaded multiprocessor with eight scalar processor (SP) cores. The eight SP cores each have a large multithreaded *register file* (RF) and share an instruction cache, multithreaded instruction issue unit, constant cache, two *special function units* (SFUs), interconnection network, and a multibank shared memory.

The pipelined SFUs execute thread instructions that compute special functions and interpolate pixel attributes from primitive vertex attributes. These instructions can execute concurrently with instructions on the SPs. The SFU is described later.

The multiprocessor executes texture fetch instructions on the texture unit via the texture interface, and uses the memory interface for external memory load, store, and atomic access instructions. These instructions can execute concurrently with instructions on the SPs. Shared memory access uses a low-latency interconnection network between the SP processors and the shared memory banks.

Single-Instruction Multiple-Thread (SIMT)

To manage and execute hundreds of threads running several different programs efficiently, the multiprocessor employs a **single-instruction multiple-thread (SIMT)** architecture. It creates, manages, schedules, and executes concurrent threads in groups of parallel threads called *warps*. The term **warp** originates from weaving, the first parallel thread technology. The photograph in Figure B.4.2 shows a warp of parallel threads emerging from a loom. This example multiprocessor uses a SIMT warp size of 32 threads, executing four threads in each of the eight SP cores over four

single-instruction multiple-thread (SIMT)

A processor architecture that applies one instruction to multiple independent threads in parallel.

warp The set of parallel threads that execute the same instruction together in a SIMT architecture.

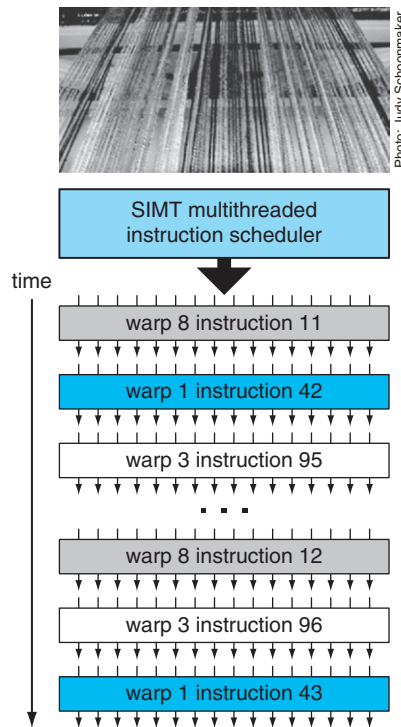


FIGURE B.4.2 SIMT multithreaded warp scheduling. The scheduler selects a ready warp and issues an instruction synchronously to the parallel threads composing the warp. Because warps are independent, the scheduler may select a different warp each time.

clocks. The Tesla SM multiprocessor described in Section B.7 also uses a warp size of 32 parallel threads, executing four threads per SP core for efficiency on plentiful pixel threads and computing threads. Thread blocks consist of one or more warps.

This example SIMT multiprocessor manages a pool of 16 warps, a total of 512 threads. Individual parallel threads composing a warp are the same type and start together at the same program address, but are otherwise free to branch and execute independently. At each instruction issue time, the SIMT multithreaded instruction unit selects a warp that is ready to execute its next instruction, and then issues that instruction to the active threads of that warp. A SIMT instruction is broadcast synchronously to the active parallel threads of a warp; individual threads may be inactive due to independent branching or predication. In this multiprocessor, each SP scalar processor core executes an instruction for four individual threads of a warp using four clocks, reflecting the 4:1 ratio of warp threads to cores.

SIMT processor architecture is akin to *single-instruction multiple data* (SIMD) design, which applies one instruction to multiple data lanes, but differs in that SIMT applies one instruction to multiple independent threads in parallel, not just

to multiple data lanes. An instruction for a SIMD processor controls a vector of multiple data lanes together, whereas an instruction for a SIMT processor controls an individual thread, and the SIMT instruction unit issues an instruction to a warp of independent parallel threads for efficiency. The SIMT processor finds data-level parallelism among threads at runtime, analogous to the way a superscalar processor finds instruction-level parallelism among instructions at runtime.

A SIMT processor realizes full efficiency and performance when all threads of a warp take the same execution path. If threads of a warp diverge via a data-dependent conditional branch, execution serializes for each branch path taken, and when all paths complete, the threads converge to the same execution path. For equal length paths, a divergent if-else code block is 50% efficient. The multiprocessor uses a branch synchronization stack to manage independent threads that diverge and converge. Different warps execute independently at full speed regardless of whether they are executing common or disjoint code paths. As a result, SIMT GPUs are dramatically more efficient and flexible on branching code than earlier GPUs, as their warps are much narrower than the SIMD width of prior GPUs.

In contrast with SIMD vector architectures, SIMT enables programmers to write thread-level parallel code for individual independent threads, as well as data-parallel code for many coordinated threads. For program correctness, the programmer can essentially ignore the SIMT execution attributes of warps; however, substantial performance improvements can be realized by taking care that the code seldom requires threads in a warp to diverge. In practice, this is analogous to the role of cache lines in traditional codes: cache line size can be safely ignored when designing for correctness but must be considered in the code structure when designing for peak performance.

SIMT Warp Execution and Divergence

The SIMT approach of scheduling independent warps is more flexible than the scheduling of previous GPU architectures. A warp comprises parallel threads of the same type: vertex, geometry, pixel, or compute. The basic unit of pixel fragment shader processing is the 2-by-2 pixel quad implemented as four pixel shader threads. The multiprocessor controller packs the pixel quads into a warp. It similarly groups vertices and primitives into warps, and packs computing threads into a warp. A thread block comprises one or more warps. The SIMT design shares the instruction fetch and issue unit efficiently across parallel threads of a warp, but requires a full warp of active threads to get full performance efficiency.

This unified multiprocessor schedules and executes multiple warp types concurrently, allowing it to concurrently execute vertex and pixel warps. Its warp scheduler operates at less than the processor clock rate, because there are four thread lanes per processor core. During each scheduling cycle, it selects a warp to execute a SIMT warp instruction, as shown in [Figure B.4.2](#). An issued warp-instruction executes as four sets of eight threads over four processor cycles of throughput. The processor pipeline uses several clocks of latency to complete each instruction. If the number of active warps times the clocks per warp exceeds the pipeline

latency, the programmer can ignore the pipeline latency. For this multiprocessor, a round-robin schedule of eight warps has a period of 32 cycles between successive instructions for the same warp. If the program can keep 256 threads active per multiprocessor, instruction latencies up to 32 cycles can be hidden from an individual sequential thread. However, with few active warps, the processor pipeline depth becomes visible and may cause processors to stall.

A challenging design problem is implementing zero-overhead warp scheduling for a dynamic mix of different warp programs and program types. The instruction scheduler must select a warp every four clocks to issue one instruction per clock per thread, equivalent to an IPC of 1.0 per processor core. Because warps are independent, the only dependences are among sequential instructions from the same warp. The scheduler uses a register dependency scoreboard to qualify warps whose active threads are ready to execute an instruction. It prioritizes all such ready warps and selects the highest priority one for issue. Prioritization must consider warp type, instruction type, and the desire to be fair to all active warps.

Managing Threads and Thread Blocks

The multiprocessor controller and instruction unit manage threads and thread blocks. The controller accepts work requests and input data and arbitrates access to shared resources, including the texture unit, memory access path, and I/O paths. For graphics workloads, it creates and manages three types of graphics threads concurrently: vertex, geometry, and pixel. Each of the graphics work types has independent input and output paths. It accumulates and packs each of these input work types into SIMT warps of parallel threads executing the same thread program. It allocates a free warp, allocates registers for the warp threads, and starts warp execution in the multiprocessor. Every program declares its per-thread register demand; the controller starts a warp only when it can allocate the requested register count for the warp threads. When all the threads of the warp exit, the controller unpacks the results and frees the warp registers and resources.

The controller creates **cooperative thread arrays (CTAs)** which implement CUDA thread blocks as one or more warps of parallel threads. It creates a CTA when it can create all CTA warps and allocate all CTA resources. In addition to threads and registers, a CTA requires allocating shared memory and barriers. The program declares the required capacities, and the controller waits until it can allocate those amounts before launching the CTA. Then it creates CTA warps at the warp scheduling rate, so that a CTA program starts executing immediately at full multiprocessor performance. The controller monitors when all threads of a CTA have exited, and frees the CTA shared resources and its warp resources.

cooperative thread array (CTA) A set of concurrent threads that executes the same thread program and may cooperate to compute a result. A GPU CTA implements a CUDA thread block.

Thread Instructions

The SP thread processors execute scalar instructions for individual threads, unlike earlier GPU vector instruction architectures, which executed four-component vector instructions for each vertex or pixel shader program. Vertex programs

generally compute (x, y, z, w) position vectors, while pixel shader programs compute (red, green, blue, alpha) color vectors. However, shader programs are becoming longer and more scalar, and it is increasingly difficult to fully occupy even two components of a legacy GPU four-component vector architecture. In effect, the SIMT architecture parallelizes across 32 independent pixel threads, rather than parallelizing the four vector components within a pixel. CUDA C/C++ programs have predominantly scalar code per thread. Previous GPUs employed vector packing (e.g., combining subvectors of work to gain efficiency) but that complicated the scheduling hardware as well as the compiler. Scalar instructions are simpler and compiler-friendly. Texture instructions remain vector-based, taking a source coordinate vector and returning a filtered color vector.

To support multiple GPUs with different binary microinstruction formats, high-level graphics and computing language compilers generate intermediate assembler-level instructions (e.g., Direct3D vector instructions or PTX scalar instructions), which are then optimized and translated to binary GPU microinstructions. The NVIDIA PTX (parallel thread execution) instruction set definition [2007] provides a stable target ISA for compilers, and provides compatibility over several generations of GPUs with evolving binary microinstruction-set architectures. The optimizer readily expands Direct3D vector instructions to multiple scalar binary microinstructions. PTX scalar instructions translate nearly one to one with scalar binary microinstructions, although some PTX instructions expand to multiple binary microinstructions, and multiple PTX instructions may fold into one binary microinstruction. Because the intermediate assembler-level instructions use virtual registers, the optimizer analyzes data dependencies and allocates real registers. The optimizer eliminates dead code, folds instructions together when feasible, and optimizes SIMT branch diverge and converge points.

Instruction Set Architecture (ISA)

The thread ISA described here is a simplified version of the Tesla architecture PTX ISA, a register-based scalar instruction set comprising floating-point, integer, logical, conversion, special functions, flow control, memory access, and texture operations. [Figure B.4.3](#) lists the basic PTX GPU thread instructions; see the NVIDIA PTX specification [2007] for details. The instruction format is:

```
opcode.type d, a, b, c;
```

where d is the destination operand, a, b, c are source operands, and .type is one of:

Type	.type Specifier
Untyped bits 8, 16, 32, and 64 bits	.b8, .b16, .b32, .b64
Unsigned integer 8, 16, 32, and 64 bits	.u8, .u16, .u32, .u64
Signed integer 8, 16, 32, and 64 bits	.s8, .s16, .s32, .s64
Floating-point 16, 32, and 64 bits	.f16, .f32, .f64

Basic PTX GPU Thread Instructions

Group	Instruction	Example	Meaning	Comments
Arithmetic	arithmetic .type = .s32, .u32, .f32, .s64, .u64, .f64			
	add.type	add.f32 d, a, b	d = a + b;	
	sub.type	sub.f32 d, a, b	d = a - b;	
	mul.type	mul.f32 d, a, b	d = a * b;	
	mad.type	mad.f32 d, a, b, c	d = a * b + c;	multiply-add
	div.type	div.f32 d, a, b	d = a / b;	multiple microinstructions
	rem.type	rem.u32 d, a, b	d = a % b;	integer remainder
	abs.type	abs.f32 d, a	d = a ;	
	neg.type	neg.f32 d, a	d = 0 - a;	
	min.type	min.f32 d, a, b	d = (a < b)? a:b;	floating selects non-NaN
	max.type	max.f32 d, a, b	d = (a > b)? a:b;	floating selects non-NaN
	setp.cmp.type	setp.lt.f32 p, a, b	p = (a < b);	compare and set predicate
	numeric .cmp = eq, ne, lt, le, gt, ge; unordered cmp = equ, neu, ltu, leu, gtu, geu, num, nan			
	mov.type	mov.b32 d, a	d = a;	move
Special Function	selp.type	selp.f32 d, a, b, p	d = p? a: b;	select with predicate
	cvt.dtype.atype	cvt.f32.s32 d, a	d = convert(a);	convert atype to dtype
	special .type = .f32 (some .f64)			
	rcp.type	rcp.f32 d, a	d = 1/a;	reciprocal
	sqrt.type	sqrt.f32 d, a	d = sqrt(a);	square root
	rsqrt.type	rsqrt.f32 d, a	d = 1/sqrt(a);	reciprocal square root
	sin.type	sin.f32 d, a	d = sin(a);	sine
	cos.type	cos.f32 d, a	d = cos(a);	cosine
Logical	lg2.type	lg2.f32 d, a	d = log(a)/log(2)	binary logarithm
	ex2.type	ex2.f32 d, a	d = 2 ** a;	binary exponential
	logic.type = .pred, .b32, .b64			
	and.type	and.b32 d, a, b	d = a & b;	
	or.type	or.b32 d, a, b	d = a b;	
	xor.type	xor.b32 d, a, b	d = a ^ b;	
	not.type	not.b32 d, a, b	d = ~a;	one's complement
	cnot.type	cnot.b32 d, a, b	d = (a==0)? 1:0;	C logical not
Memory Access	shl.type	shl.b32 d, a, b	d = a << b;	shift left
	shr.type	shr.s32 d, a, b	d = a >> b;	shift right
	memory .space = .global, .shared, .local, .const; .type = .b8, .u8, .s8, .b16, .b32, .b64			
	ld.space.type	ld.global.b32 d, [a+off]	d = *(a+off);	load from memory space
	st.space.type	st.shared.b32 [d+off], a	*(d+off) = a;	store to memory space
	tex.nd.dtyp.btype	tex.2d.v4.f32.f32 d, a, b	d = tex2d(a, b);	texture lookup
	atom.spc.op.type	atom.global.add.u32 d,[a], b atom.global.cas.b32 d,[a], b, c	atomic { d = *a; *a = op(*a, b); }	atomic read-modify-write operation
Control Flow	atom.op = and, or, xor, add, min, max, exch, cas; .spc = .global; .type = .b32			
	branch	@p bra target	if (p) goto target;	conditional branch
	call	call (ret), func, (params)	ret = func(params);	call function
	ret	ret	return;	return from function call
	bar.sync	bar.sync d	wait for threads	barrier synchronization
	exit	exit	exit;	terminate thread execution

FIGURE B.4.3 Basic PTX GPU thread instructions.

Source operands are scalar 32-bit or 64-bit values in registers, an immediate value, or a constant; predicate operands are 1-bit Boolean values. Destinations are registers, except for store to memory. Instructions are predicated by prefixing them with `@p` or `!p`, where `p` is a predicate register. Memory and texture instructions transfer scalars or vectors of two to four components, up to 128 bits in total. PTX instructions specify the behavior of one thread.

The PTX arithmetic instructions operate on 32-bit and 64-bit floating-point, signed integer, and unsigned integer types. Recent GPUs support 64-bit double-precision floating-point; see Section B.6. On current GPUs, PTX 64-bit integer and logical instructions are translated to two or more binary microinstructions that perform 32-bit operations. The GPU special function instructions are limited to 32-bit floating-point. The thread control flow instructions are conditional branch, function call and return, thread exit, and `bar.sync` (barrier synchronization). The conditional branch instruction `@p bra target` uses a predicate register `p` (or `!p`) previously set by a compare and set predicate `setp` instruction to determine whether the thread takes the branch or not. Other instructions can also be predicated on a predicate register being true or false.

Memory Access Instructions

The `tex` instruction fetches and filters texture samples from 1D, 2D, and 3D texture arrays in memory via the texture subsystem. Texture fetches generally use interpolated floating-point coordinates to address a texture. Once a graphics pixel shader thread computes its pixel fragment color, the raster operations processor blends it with the pixel color at its assigned (x, y) pixel position and writes the final color to memory.

To support computing and C/C++ language needs, the Tesla PTX ISA implements memory load/store instructions. It uses integer byte addressing with register plus offset address arithmetic to facilitate conventional compiler code optimizations. Memory load/store instructions are common in processors, but are a significant new capability in the Tesla architecture GPUs, as prior GPUs provided only the texture and pixel accesses required by the graphics APIs.

For computing, the load/store instructions access three read/write memory spaces that implement the corresponding CUDA memory spaces in Section B.3:

- Local memory for per-thread private addressable temporary data (implemented in external DRAM)
- Shared memory for low-latency access to data shared by cooperating threads in the same CTA/thread block (implemented in on-chip SRAM)
- Global memory for large data sets shared by all threads of a computing application (implemented in external DRAM)

The memory load/store instructions `ld.global`, `st.global`, `ld.shared`, `st.shared`, `ld.local`, and `st.local` access the global, shared, and local memory spaces. Computing programs use the fast barrier synchronization instruction `bar.sync` to synchronize threads within a CTA/thread block that communicate with each other via shared and global memory.

To improve memory bandwidth and reduce overhead, the local and global load/store instructions coalesce individual parallel thread requests from the same SIMT warp together into a single memory block request when the addresses fall in the same block and meet alignment criteria. Coalescing memory requests provides a significant performance boost over separate requests from individual threads. The multiprocessor's large thread count, together with support for many outstanding load requests, helps cover load-to-use latency for local and global memory implemented in external DRAM.

The latest Tesla architecture GPUs also provide efficient atomic memory operations on memory with the `atom.op.u32` instructions, including integer operations `add`, `min`, `max`, `and`, `or`, `xor`, `exchange`, and `cas` (compare-and-swap) operations, facilitating parallel reductions and parallel data structure management.

Barrier Synchronization for Thread Communication

Fast barrier synchronization permits CUDA programs to communicate frequently via shared memory and global memory by simply calling `__syncthreads()`; as part of each interthread communication step. The synchronization intrinsic function generates a single `bar.sync` instruction. However, implementing fast barrier synchronization among up to 512 threads per CUDA thread block is a challenge.

Grouping threads into SIMT warps of 32 threads reduces the synchronization difficulty by a factor of 32. Threads wait at a barrier in the SIMT thread scheduler so they do not consume any processor cycles while waiting. When a thread executes a `bar.sync` instruction, it increments the barrier's thread arrival counter and the scheduler marks the thread as waiting at the barrier. Once all the CTA threads arrive, the barrier counter matches the expected terminal count, and the scheduler releases all the threads waiting at the barrier and resumes executing threads.

Streaming Processor (SP)

The multithreaded *streaming processor* (SP) core is the primary thread instruction processor in the multiprocessor. Its *register file* (RF) provides 1024 scalar 32-bit registers for up to 64 threads. It executes all the fundamental floating-point operations, including `add.f32`, `mul.f32`, `mad.f32` (floating multiply-add), `min.f32`, `max.f32`, and `setp.f32` (floating compare and set predicate). The floating-point add and multiply operations are compatible with the IEEE 754 standard for single-precision FP numbers, including *not-a-number* (NaN) and infinity values. The SP core also implements all of the 32-bit and 64-bit integer arithmetic, comparison, conversion, and logical PTX instructions shown in [Figure B.4.3](#).

The floating-point `add` and `mul` operations employ IEEE round-to-nearest-even as the default rounding mode. The `mad.f32` floating-point multiply-add operation performs a multiplication with truncation, followed by an addition with round-to-nearest-even. The SP flushes input denormal operands to sign-preserved-zero. Results that underflow the target output exponent range are flushed to sign-preserved-zero after rounding.

Special Function Unit (SFU)

Certain thread instructions can execute on the SFUs, concurrently with other thread instructions executing on the SPs. The SFU implements the special function instructions of [Figure B.4.3](#), which compute 32-bit floating-point approximations to reciprocal, reciprocal square root, and key transcendental functions. It also implements 32-bit floating-point planar attribute interpolation for pixel shaders, providing accurate interpolation of attributes such as color, depth, and texture coordinates.

Each pipelined SFU generates one 32-bit floating-point special function result per cycle; the two SFUs per multiprocessor execute special function instructions at a quarter the simple instruction rate of the eight SPs. The SFUs also execute the `mul.f32` multiply instruction concurrently with the eight SPs, increasing the peak computation rate up to 50% for threads with a suitable instruction mixture.

For functional evaluation, the Tesla architecture SFU employs quadratic interpolation based on enhanced minimax approximations for approximating the reciprocal, reciprocal square-root, $\log_2 x$, $2x$, and $\sin/\cos$ functions. The accuracy of the function estimates ranges from 22 to 24 mantissa bits. See Section B.6 for more details on SFU arithmetic.

Comparing with Other Multiprocessors

Compared with SIMD vector architectures such as x86 SSE, the SIMT multiprocessor can execute individual threads independently, rather than always executing them together in synchronous groups. SIMT hardware finds data parallelism among independent threads, whereas SIMD hardware requires the software to express data parallelism explicitly in each vector instruction. A SIMT machine executes a warp of 32 threads synchronously when the threads take the same execution path, yet can execute each thread independently when they diverge. The advantage is significant because SIMT programs and instructions simply describe the behavior of a single independent thread, rather than a SIMD data vector of four or more data lanes. Yet the SIMT multiprocessor has SIMD-like efficiency, spreading the area and cost of one instruction unit across the 32 threads of a warp and across the eight streaming processor cores. SIMT provides the performance of SIMD together with the productivity of multithreading, avoiding the need to explicitly code SIMD vectors for edge conditions and partial divergence.

The SIMT multiprocessor imposes little overhead because it is hardware multithreaded with hardware barrier synchronization. That allows graphics shaders and CUDA threads to express very fine-grained parallelism. Graphics and CUDA programs use threads to express fine-grained data parallelism in a per-thread program, rather than forcing the programmer to express it as SIMD vector instructions. It is simpler and more productive to develop scalar single-thread code than vector code, and the SIMT multiprocessor executes the code with SIMD-like efficiency.

Coupling eight streaming processor cores together closely into a multiprocessor and then implementing a scalable number of such multiprocessors makes a two-level multiprocessor composed of multiprocessors. The CUDA programming model exploits the two-level hierarchy by providing individual threads for fine-grained parallel computations, and by providing grids of thread blocks for coarse-grained parallel operations. The same thread program can provide both fine-grained and coarse-grained operations. In contrast, CPUs with SIMD vector instructions must use two different programming models to provide fine-grained and coarse-grained operations: coarse-grained parallel threads on different cores, and SIMD vector instructions for fine-grained data parallelism.

Multithreaded Multiprocessor Conclusion

The example GPU multiprocessor based on the Tesla architecture is highly multithreaded, executing a total of up to 512 lightweight threads concurrently to support fine-grained pixel shaders and CUDA threads. It uses a variation on SIMD architecture and multithreading called SIMT (*single-instruction multiple-thread*) to efficiently broadcast one instruction to a warp of 32 parallel threads, while permitting each thread to branch and execute independently. Each thread executes its instruction stream on one of the eight *streaming processor* (SP) cores, which are multithreaded up to 64 threads.

The PTX ISA is a register-based load/store scalar ISA that describes the execution of a single thread. Because PTX instructions are optimized and translated to binary microinstructions for a specific GPU, the hardware instructions can evolve rapidly without disrupting compilers and software tools that generate PTX instructions.

B.5 Parallel Memory System

Outside of the GPU itself, the memory subsystem is the most important determiner of the performance of a graphics system. Graphics workloads demand very high transfer rates to and from memory. Pixel write and blend (read-modify-write) operations, depth buffer reads and writes, and texture map reads, as well as command and object vertex and attribute data reads, comprise the majority of memory traffic.

Modern GPUs are highly parallel, as shown in [Figure B.2.5](#). For example, the GeForce 8800 can process 32 pixels per clock, at 600 MHz. Each pixel typically requires a color read and write and a depth read and write of a 4-byte pixel. Usually an average of two or three texels of four bytes each are read to generate the pixel's color. So for a typical case, there is a demand of 28 bytes times 32 pixels = 896 bytes per clock. Clearly the bandwidth demand on the memory system is enormous.

To supply these requirements, GPU memory systems have the following characteristics:

- They are wide, meaning there are a large number of pins to convey data between the GPU and its memory devices, and the memory array itself comprises many DRAM chips to provide the full total data bus width.
- They are fast, meaning aggressive signaling techniques are used to maximize the data rate (bits/second) per pin.
- GPUs seek to use every available cycle to transfer data to or from the memory array. To achieve this, GPUs specifically do not aim to minimize latency to the memory system. High throughput (utilization efficiency) and short latency are fundamentally in conflict.
- Compression techniques are used, both lossy, of which the programmer must be aware, and lossless, which is invisible to the application and opportunistic.
- Caches and work coalescing structures are used to reduce the amount of off-chip traffic needed and to ensure that cycles spent moving data are used as fully as possible.

DRAM Considerations

GPUs must take into account the unique characteristics of DRAM. DRAM chips are internally arranged as multiple (typically four to eight) banks, where each bank includes a power-of-2 number of rows (typically around 16,384), and each row contains a power-of-2 number of bits (typically 8192). DRAMs impose a variety of timing requirements on their controlling processor. For example, dozens of cycles are required to activate one row, but once activated, the bits within that row are randomly accessible with a new column address every four clocks. *Double-data rate* (DDR) synchronous DRAMs transfer data on both rising and falling edges of the interface clock (see [Chapter 5](#)). So a 1 GHz clocked DDR DRAM transfers data at 2 gigabits per second per data pin. Graphics DDR DRAMs usually have 32 bidirectional data pins, so eight bytes can be read or written from the DRAM per clock.

GPUs internally have a large number of generators of memory traffic. Different stages of the logical graphics pipeline each have their own request streams: command and vertex attribute fetch, shader texture fetch and load/store, and pixel depth and color read-write. At each logical stage, there are often multiple independent units to deliver the parallel throughput. These are each independent memory requestors. When viewed at the memory system, there is an enormous number of uncorrelated requests in flight. This is a natural mismatch to the reference pattern preferred by the DRAMs. A solution is for the GPU's memory controller to maintain separate heaps of traffic bound for different DRAM banks, and wait until enough traffic for

a particular DRAM row is pending before activating that row and transferring all the traffic at once. Note that accumulating pending requests, while good for DRAM row locality and thus efficient use of the data bus, leads to longer average latency as seen by the requestors whose requests spend time waiting for others. The design must take care that no particular request waits too long, otherwise some processing units can starve waiting for data and ultimately cause neighboring processors to become idle.

GPU memory subsystems are arranged as multiple *memory partitions*, each of which comprises a fully independent memory controller and one or two DRAM devices that are fully and exclusively owned by that partition. To achieve the best load balance and therefore approach the theoretical performance of n partitions, addresses are finely interleaved evenly across all memory partitions. The partition interleaving stride is typically a block of a few hundred bytes. The number of memory partitions is designed to balance the number of processors and other memory requesters.

Caches

GPU workloads typically have very large working sets—on the order of hundreds of megabytes to generate a single graphics frame. Unlike with CPUs, it is not practical to construct caches on chips large enough to hold anything close to the full working set of a graphics application. Whereas CPUs can assume very high cache hit rates (99.9% or more), GPUs experience hit rates closer to 90% and must therefore cope with many misses in flight. While a CPU can reasonably be designed to halt while waiting for a rare cache miss, a GPU needs to proceed with misses and hits intermingled. We call this a *streaming cache architecture*.

GPU caches must deliver very high-bandwidth to their clients. Consider the case of a texture cache. A typical texture unit may evaluate two bilinear interpolations for each of four pixels per clock cycle, and a GPU may have many such texture units all operating independently. Each bilinear interpolation requires four separate texels, and each texel might be a 64-bit value. Four 16-bit components are typical. Thus, total bandwidth is $2 \times 4 \times 4 \times 64 = 2048$ bits per clock. Each separate 64-bit texel is independently addressed, so the cache needs to handle 32 unique addresses per clock. This naturally favors a multibank and/or multiport arrangement of SRAM arrays.

MMU

Modern GPUs are capable of translating virtual addresses to physical addresses. On the GeForce 8800, all processing units generate memory addresses in a 40-bit virtual address space. For computing, load and store thread instructions use 32-bit byte addresses, which are extended to a 40-bit virtual address by adding a 40-bit offset. A memory management unit performs virtual to physical address

translation; hardware reads the page tables from local memory to respond to misses on behalf of a hierarchy of translation lookaside buffers spread out among the processors and rendering engines. In addition to physical page bits, GPU page table entries specify the compression algorithm for each page. Page sizes range from 4 to 128 kilobytes.

Memory Spaces

As introduced in Section B.3, CUDA exposes different memory spaces to allow the programmer to store data values in the most performance-optimal way. For the following discussion, NVIDIA Tesla architecture GPUs are assumed.

Global memory

Global memory is stored in external DRAM; it is not local to any one physical *streaming multiprocessor* (SM) because it is meant for communication among different CTAs (thread blocks) in different grids. In fact, the many CTAs that reference a location in global memory may not be executing in the GPU at the same time; by design, in CUDA a programmer does not know the relative order in which CTAs are executed. Because the address space is evenly distributed among all memory partitions, there must be a read/write path from any streaming multiprocessor to any DRAM partition.

Access to global memory by different threads (and different processors) is not guaranteed to have sequential consistency. Thread programs see a relaxed memory ordering model. Within a thread, the order of memory reads and writes to the same address is preserved, but the order of accesses to different addresses may not be preserved. Memory reads and writes requested by different threads are unordered. Within a CTA, the barrier synchronization instruction `bar.sync` can be used to obtain strict memory ordering among the threads of the CTA. The `membar` thread instruction provides a memory barrier/fence operation that commits prior memory accesses and makes them visible to other threads before proceeding. Threads can also use the atomic memory operations described in Section B.4 to coordinate work on memory they share.

Shared memory

Per-CTA shared memory is only visible to the threads that belong to that CTA, and shared memory only occupies storage from the time a CTA is created to the time it terminates. Shared memory can therefore reside on-chip. This approach has many benefits. First, shared memory traffic does not need to compete with limited off-chip bandwidth needed for global memory references. Second, it is practical to build very high-bandwidth memory structures on-chip to support the read/write demands of each streaming multiprocessor. In fact, the shared memory is closely coupled to the streaming multiprocessor.

Each streaming multiprocessor contains eight physical thread processors. During one shared memory clock cycle, each thread processor can process two threads' worth of instructions, so 16 threads' worth of shared memory requests must be handled in each clock. Because each thread can generate its own addresses, and the addresses are typically unique, the shared memory is built using 16 independently addressable SRAM banks. For common access patterns, 16 banks are sufficient to maintain throughput, but pathological cases are possible; for example, all 16 threads might happen to access a different address on one SRAM bank. It must be possible to route a request from any thread lane to any bank of SRAM, so a 16-by-16 interconnection network is required.

Local Memory

Per-thread local memory is private memory visible only to a single thread. Local memory is architecturally larger than the thread's register file, and a program can compute addresses into local memory. To support large allocations of local memory (recall the total allocation is the per-thread allocation times the number of active threads), local memory is allocated in external DRAM.

Although global and per-thread local memory reside off-chip, they are well-suited to being cached on-chip.

Constant Memory

Constant memory is read-only to a program running on the SM (it can be written via commands to the GPU). It is stored in external DRAM and cached in the SM. Because commonly most or all threads in a SIMT warp read from the same address in constant memory, a single address lookup per clock is sufficient. The constant cache is designed to broadcast scalar values to threads in each warp.

Texture Memory

Texture memory holds large read-only arrays of data. Textures for computing have the same attributes and capabilities as textures used with 3D graphics. Although textures are commonly two-dimensional images (2D arrays of pixel values), 1D (linear) and 3D (volume) textures are also available.

A compute program references a texture using a `tex` instruction. Operands include an identifier to name the texture, and one, two, or three coordinates based on the texture dimensionality. The floating-point coordinates include a fractional portion that specifies a sample location, often in-between texel locations. Noninteger coordinates invoke a bilinear weighted interpolation of the four closest values (for a 2D texture) before the result is returned to the program.

Texture fetches are cached in a streaming cache hierarchy designed to optimize throughput of texture fetches from thousands of concurrent threads. Some programs use texture fetches as a way to cache global memory.

Surfaces

Surface is a generic term for a one-dimensional, two-dimensional, or three-dimensional array of pixel values and an associated format. A variety of formats are defined; for example, a pixel may be defined as four 8-bit RGBA integer components, or four 16-bit floating-point components. A program kernel does not need to know the surface type. A `tex` instruction recasts its result values as floating-point, depending on the surface format.

Load/Store Access

Load/store instructions with integer byte addressing enable the writing and compiling of programs in conventional languages like C and C++. CUDA programs use load/store instructions to access memory.

To improve memory bandwidth and reduce overhead, the local and global load/store instructions coalesce individual parallel thread requests from the same warp together into a single memory block request when the addresses fall in the same block and meet alignment criteria. Coalescing individual small memory requests into large block requests provides a significant performance boost over separate requests. The large thread count, together with support for many outstanding load requests, helps cover load-to-use latency for local and global memory implemented in external DRAM.

ROP

As shown in [Figure B.2.5](#), NVIDIA Tesla architecture GPUs comprise a scalable *streaming processor array* (SPA), which performs all of the GPU's programmable calculations, and a scalable memory system, which comprises external DRAM control and fixed function *Raster Operation Processors* (ROPs) that perform color and depth framebuffer operations directly on memory. Each ROP unit is paired with a specific memory partition. ROP partitions are fed from the SMs via an interconnection network. Each ROP is responsible for depth and stencil tests and updates, as well as color blending. The ROP and memory controllers cooperate to implement lossless color and depth compression (up to 8:1) to reduce external bandwidth demand. ROP units also perform atomic operations on memory.

B.6

Floating-point Arithmetic

GPUs today perform most arithmetic operations in the programmable processor cores using IEEE 754-compatible single precision 32-bit floating-point operations (see [Chapter 3](#)). The fixed-point arithmetic of early GPUs was succeeded by 16-bit, 24-bit, and 32-bit floating-point, then IEEE 754-compatible 32-bit floating-point.

Some fixed-function logic within a GPU, such as texture-filtering hardware, continues to use proprietary numeric formats. Recent GPUs also provide IEEE 754-compatible double-precision 64-bit floating-point instructions.

Supported Formats

The IEEE 754 standard for floating-point arithmetic specifies basic and storage formats. GPUs use two of the basic formats for computation, 32-bit and 64-bit binary floating-point, commonly called single precision and double precision. The standard also specifies a 16-bit binary storage floating-point format, **half precision**. GPUs and the Cg shading language employ the narrow 16-bit half data format for efficient data storage and movement, while maintaining high dynamic range. GPUs perform many texture filtering and pixel blending computations at half precision within the texture filtering unit and the raster operations unit. The OpenEXR high dynamic-range image file format developed by [Industrial Light and Magic \[2003\]](#) uses the identical half format for color component values in computer imaging and motion picture applications.

half precision A 16-bit binary floating-point format, with 1 sign bit, 5-bit exponent, 10-bit fraction, and an implied integer bit.

Basic Arithmetic

Common single-precision floating-point operations in GPU programmable cores include addition, multiplication, **multiply-add**, minimum, maximum, compare, set predicate, and conversions between integer and floating-point numbers. Floating-point instructions often provide source operand modifiers for negation and absolute value.

multiply-add (MAD)
A single floating-point instruction that performs a compound operation: multiplication followed by addition.

The floating-point addition and multiplication operations of most GPUs today are compatible with the IEEE 754 standard for single precision FP numbers, including *not-a-number* (NaN) and infinity values. The FP addition and multiplication operations use IEEE round-to-nearest-even as the default rounding mode. To increase floating-point instruction throughput, GPUs often use a compound multiply-add instruction (`mad`). The multiply-add operation performs FP multiplication with truncation, followed by FP addition with round-to-nearest-even. It provides two floating-point operations in one issuing cycle, without requiring the instruction scheduler to dispatch two separate instructions, but the computation is not fused and truncates the product before the addition. This makes it different from the fused multiply-add instruction discussed in [Chapter 3](#) and later in this section. GPUs typically flush denormalized source operands to sign-preserved zero, and they flush results that underflow the target output exponent range to sign-preserved zero after rounding.

Specialized Arithmetic

GPUs provide hardware to accelerate special function computation, attribute interpolation, and texture filtering. Special function instructions include cosine,

sine, binary exponential, binary logarithm, reciprocal, and reciprocal square root. Attribute interpolation instructions provide efficient generation of pixel attributes, derived from plane equation evaluation. The **special function unit (SFU)** introduced in Section B.4 computes special functions and interpolates planar attributes [Oberman and Siu, 2005].

Several methods exist for evaluating special functions in hardware. It has been shown that quadratic interpolation based on Enhanced Minimax Approximations is a very efficient method for approximating functions in hardware, including reciprocal, reciprocal square-root, $\log_2 x$, 2^x , sin, and cos.

We can summarize the method of SFU quadratic interpolation. For a binary input operand X with n -bit significand, the significand is divided into two parts: X_u is the upper part containing m bits, and X_l is the lower part containing $n-m$ bits. The upper m bits X_u are used to consult a set of three lookup tables to return three finite-word coefficients C_0 , C_1 , and C_2 . Each function to be approximated requires a unique set of tables. These coefficients are used to approximate a given function $f(X)$ in the range $X_u \leq X < X_u + 2^{-m}$ by evaluating the expression:

$$f(X) = C_0 + C_1 X_l + C_2 X_l^2$$

The accuracy of each of the function estimates ranges from 22 to 24 significand bits. Example function statistics are shown in Figure B.6.1.

The IEEE 754 standard specifies exact-rounding requirements for division and square root; however, for many GPU applications, exact compliance is not required. Rather, for those applications, higher computational throughput is more important than last-bit accuracy. For the SFU special functions, the CUDA math library provides both a full accuracy function and a fast function with the SFU instruction accuracy.

Another specialized arithmetic operation in a GPU is attribute interpolation. Key *attributes* are usually specified for vertices of primitives that make up a scene to be rendered. Example attributes are color, depth, and texture coordinates. These attributes must be interpolated in the (x,y) screen space as needed to determine the

special function unit (SFU) A hardware unit that computes special functions and interpolates planar attributes.

Function	Input interval	Accuracy (good bits)	ULP* error	% exactly rounded	Monotonic
$1/x$	[1, 2)	24.02	0.98	87	Yes
$1/\text{sqrt}(x)$	[1, 4)	23.40	1.52	78	Yes
2^x	[0, 1)	22.51	1.41	74	Yes
$\log_2 x$	[1, 2)	22.57	N/A**	N/A	Yes
sin/cos	[0, $\pi/2$)	22.47	N/A	N/A	No

*ULP: unit in the last place.**N/A: not applicable.

FIGURE B.6.1 Special function approximation statistics. For the NVIDIA GeForce 8800 *special function unit (SFU)*.

values of the attributes at each pixel location. The value of a given attribute U in an (x, y) plane can be expressed using plane equations of the form:

$$U(x,y) = A_u x + B_u y + C_u$$

where A , B , and C are interpolation parameters associated with each attribute U . The interpolation parameters A , B , and C are all represented as single-precision floating-point numbers.

Given the need for both a function evaluator and an attribute interpolator in a pixel shader processor, a single SFU that performs both functions for efficiency can be designed. Both functions use a sum of products operation to interpolate results, and the number of terms to be summed in both functions is very similar.

Texture Operations

Texture mapping and filtering is another key set of specialized floating-point arithmetic operations in a GPU. The operations used for texture mapping include:

1. Receive texture address (s, t) for the current screen pixel (x, y) , where s and t are single-precision floating-point numbers.
2. Compute the level of detail to identify the correct texture **MIP-map** level.
3. Compute the trilinear interpolation fraction.
4. Scale texture address (s, t) for the selected MIP-map level.
5. Access memory and retrieve desired texels (texture elements).
6. Perform filtering operation on texels.

MIP-map A Latin phrase *multum in parvo*, or much in a small space. A MIP-map contains precalculated images of different resolutions, used to increase rendering speed and reduce artifacts.

Texture mapping requires a significant amount of floating-point computation for full-speed operation, much of which is done at 16-bit half precision. As an example, the GeForce 8800 Ultra delivers about 500 GFLOPS of proprietary format floating-point computation for texture mapping instructions, in addition to its conventional IEEE single-precision floating-point instructions. For more details on texture mapping and filtering, see [Foley and van Dam \[1995\]](#).

Performance

The floating-point addition and multiplication arithmetic hardware is fully pipelined, and latency is optimized to balance delay and area. While pipelined, the throughput of the special functions is less than the floating-point addition and multiplication operations. Quarter-speed throughput for the special functions is typical performance in modern GPUs, with one SFU shared by four SP cores. In contrast, CPUs typically have significantly lower throughput for similar functions, such as division and square root, albeit with more accurate results. The attribute interpolation hardware is typically fully pipelined to enable full-speed pixel shaders.

Double precision

Newer GPUs such as the Tesla T10P also support IEEE 754 64-bit double-precision operations in hardware. Standard floating-point arithmetic operations in double precision include addition, multiplication, and conversions between different floating-point and integer formats. The 2008 IEEE 754 floating-point standard includes specification for the *fused-multiply-add* (FMA) operation, as discussed in [Chapter 3](#). The FMA operation performs a floating-point multiplication followed by an addition, with a single rounding. The fused multiplication and addition operations retain full accuracy in intermediate calculations. This behavior enables more accurate floating-point computations involving the accumulation of products, including dot products, matrix multiplication, and polynomial evaluation. The FMA instruction also enables efficient software implementations of exactly rounded division and square root, removing the need for a hardware division or square root unit.

A double-precision hardware FMA unit implements 64-bit addition, multiplication, conversions, and the FMA operation itself. The architecture of a

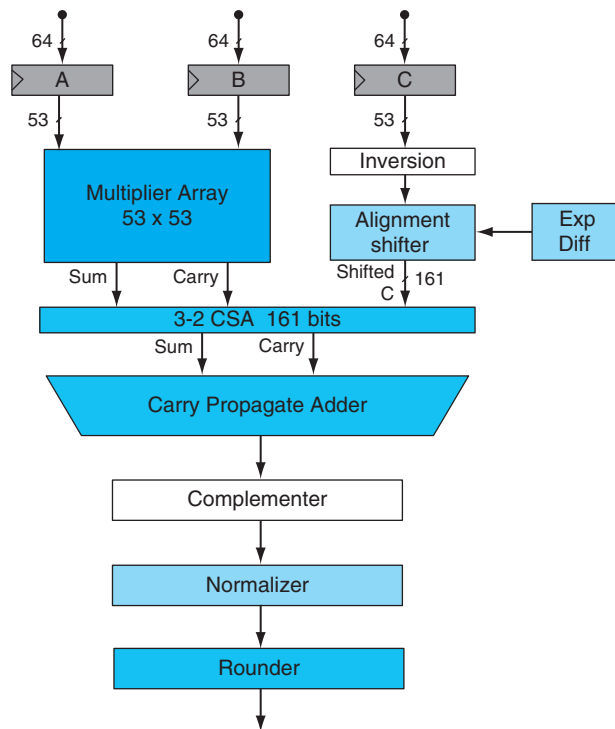


FIGURE B.6.2 Double-precision fused-multiply-add (FMA) unit. Hardware to implement floating-point $A \times B + C$ for double precision.

double-precision FMA unit enables full-speed denormalized number support on both inputs and outputs. [Figure B.6.2](#) shows a block diagram of an FMA unit.

As shown in [Figure B.6.2](#), the significands of A and B are multiplied to form a 106-bit product, with the results left in carry-save form. In parallel, the 53-bit addend C is conditionally inverted and aligned to the 106-bit product. The sum and carry results of the 106-bit product are summed with the aligned addend through a 161-bit-wide *carry-save adder* (CSA). The carry-save output is then summed together in a carry-propagate adder to produce an unrounded result in nonredundant, two's complement form. The result is conditionally recomplemented, so as to return a result in sign-magnitude form. The complemented result is normalized, and then it is rounded to fit within the target format.

B.7

Real Stuff: The NVIDIA GeForce 8800

The NVIDIA GeForce 8800 GPU, introduced in November 2006, is a unified vertex and pixel processor design that also supports parallel computing applications written in C using the CUDA parallel programming model. It is the first implementation of the Tesla unified graphics and computing architecture described in Section B.4 and in [Lindholm et al. \[2008\]](#). A family of Tesla architecture GPUs addresses the different needs of laptops, desktops, workstations, and servers.

Streaming Processor Array (SPA)

The GeForce 8800 GPU shown in [Figure B.7.1](#) contains 128 *streaming processor* (SP) cores organized as 16 *streaming multiprocessors* (SMs). Two SMs share a texture unit in each *texture/processor cluster* (TPC). An array of eight TPCs makes up the *streaming processor array* (SPA), which executes all graphics shader programs and computing programs.

The host interface unit communicates with the host CPU via the PCI-Express bus, checks command consistency, and performs context switching. The input assembler collects geometric primitives (points, lines, triangles). The work distribution blocks dispatch vertices, pixels, and compute thread arrays to the TPCs in the SPA. The TPCs execute vertex and geometry shader programs and computing programs. Output geometric data are sent to the viewport/clip/setup/raster/zcull block to be rasterized into pixel fragments that are then redistributed back into the SPA to execute pixel shader programs. Shaded pixels are sent across the interconnection network for processing by the ROP units. The network also routes texture memory read requests from the SPA to DRAM and reads data from DRAM through a level-2 cache back to the SPA.

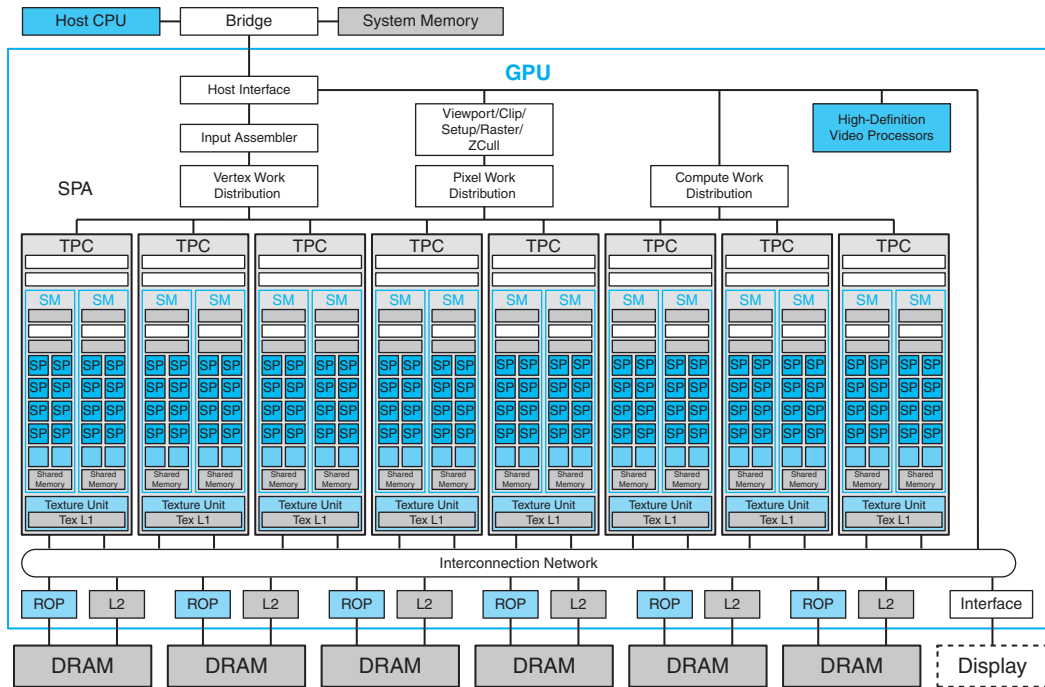


FIGURE B.7.1 NVIDIA Tesla unified graphics and computing GPU architecture. This GeForce 8800 has 128 *streaming processor* (SP) cores in 16 *streaming multiprocessors* (SMs), arranged in eight *texture/processor clusters* (TPCs). The processors connect with six 64-bit-wide DRAM partitions via an interconnection network. Other GPUs implementing the Tesla architecture vary the number of SP cores, SMs, DRAM partitions, and other units.

Texture/Processor Cluster (TPC)

Each TPC contains a geometry controller, an SMC, two SMs, and a texture unit as shown in Figure B.7.2.

The geometry controller maps the logical graphics vertex pipeline into recirculation on the physical SMs by directing all primitive and vertex attribute and topology flow in the TPC.

The SMC controls multiple SMs, arbitrating the shared texture unit, load/store path, and I/O path. The SMC serves three graphics workloads simultaneously: vertex, geometry, and pixel.

The texture unit processes a texture instruction for one vertex, geometry, or pixel quad, or four compute threads per cycle. Texture instruction sources are texture coordinates, and the outputs are weighted samples, typically a four-component (RGBA) floating-point color. The texture unit is deeply pipelined. Although it contains a streaming cache to capture filtering locality, it streams hits mixed with misses without stalling.

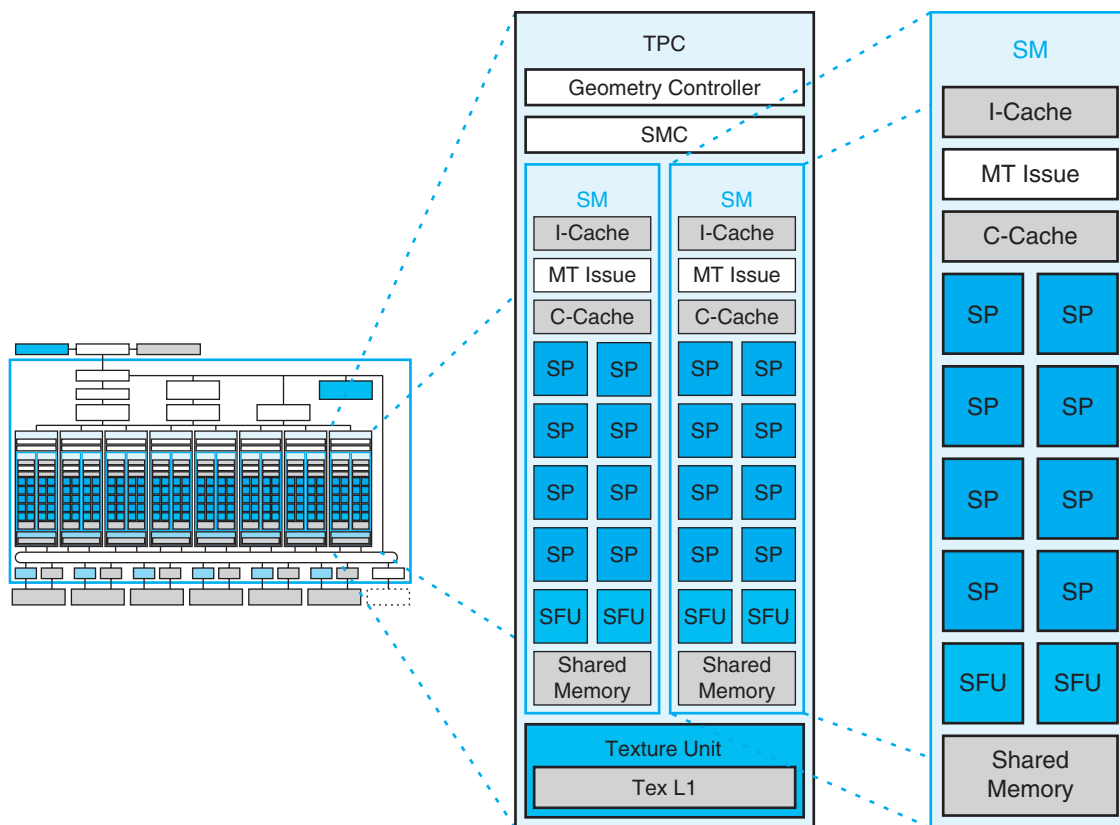


FIGURE B.7.2 Texture/processor cluster (TPC) and a streaming multiprocessor (SM). Each SM has eight *streaming processor* (SP) cores, two SFUs, and a shared memory.

Streaming Multiprocessor (SM)

The SM is a unified graphics and computing multiprocessor that executes vertex, geometry, and pixel-fragment shader programs and parallel computing programs. The SM consists of eight SP thread processor cores, two SFUs, a multithreaded instruction fetch and issue unit (MT issue), an instruction cache, a read-only constant cache, and a 16KB read/write shared memory. It executes scalar instructions for individual threads.

The GeForce 8800 Ultra clocks the SP cores and SFUs at 1.5 GHz, for a peak of 36 GFLOPS per SM. To optimize power and area efficiency, some SM nondatapath units operate at half the SP clock rate.

To efficiently execute hundreds of parallel threads while running several different programs, the SM is hardware multithreaded. It manages and executes up to 768 concurrent threads in hardware with zero scheduling overhead. Each thread has its own thread execution state and can execute an independent code path.

A warp consists of up to 32 threads of the same type—vertex, geometry, pixel, or compute. The SIMT design, previously described in Section B.4, shares the SM instruction fetch and issue unit efficiently across 32 threads but requires a full warp of active threads for full performance efficiency.

The SM schedules and executes multiple warp types concurrently. Each issue cycle, the scheduler selects one of the 24 warps to execute a SIMT warp instruction. An issued warp instruction executes as four sets of eight threads over four processor cycles. The SP and SFU units execute instructions independently, and by issuing instructions between them on alternate cycles, the scheduler can keep both fully occupied. A scoreboard qualifies each warp for issue each cycle. The instruction scheduler prioritizes all ready warps and selects the one with highest priority for issue. Prioritization considers warp type, instruction type, and “fairness” to all warps executing in the SM.

The SM executes *cooperative thread arrays* (CTAs) as multiple concurrent warps which access a shared memory region allocated dynamically for the CTA.

Instruction Set

Threads execute scalar instructions, unlike previous GPU vector instruction architectures. Scalar instructions are simpler and compiler-friendly. Texture instructions remain vector-based, taking a source coordinate vector and returning a filtered color vector.

The register-based instruction set includes all the floating-point and integer arithmetic, transcendental, logical, flow control, memory load/store, and texture instructions listed in the PTX instruction table of [Figure B.4.3](#). Memory load/store instructions use integer byte addressing with register-plus-offset address arithmetic. For computing, the load/store instructions access three read-write memory spaces: local memory for per-thread, private, temporary data; shared memory for low-latency per-CTA data shared by the threads of the CTA; and global memory for data shared by all threads. Computing programs use the fast barrier synchronization `bar.sync` instruction to synchronize threads within a CTA that communicate with each other via shared and global memory. The latest Tesla architecture GPUs implement PTX atomic memory operations, which facilitate parallel reductions and parallel data structure management.

Streaming Processor (SP)

The multithreaded SP core is the primary thread processor, as introduced in Section B.4. Its register file provides 1024 scalar 32-bit registers for up to 96 threads (more threads than in the example SP of Section B.4). Its floating-point add and

multiply operations are compatible with the IEEE 754 standard for single-precision FP numbers, including *not-a-number* (NaN) and infinity. The add and multiply operations use IEEE round-to-nearest-even as the default rounding mode. The SP core also implements all of the 32-bit and 64-bit integer arithmetic, comparison, conversion, and logical PTX instructions in [Figure B.4.3](#). The processor is fully pipelined, and latency is optimized to balance delay and area.

Special Function Unit (SFU)

The SFU supports computation of both transcendental functions and planar attribute interpolation. As described in Section B.6, it uses quadratic interpolation based on enhanced minimax approximations to approximate the reciprocal, reciprocal square root, $\log_2 x$, 2^x , and sin/cos functions at one result per cycle. The SFU also supports pixel attribute interpolation such as color, depth, and texture coordinates at four samples per cycle.

Rasterization

Geometry primitives from the SMs go in their original round-robin input order to the viewport/clip/setup/raster/zcull block. The viewport and clip units clip the primitives to the view frustum and to any enabled user clip planes, and then transform the vertices into screen (pixel) space.

Surviving primitives then go to the setup unit, which generates edge equations for the rasterizer. A coarse-rasterization stage generates all pixel tiles that are at least partially inside the primitive. The zcull unit maintains a hierarchical z surface, rejecting pixel tiles if they are conservatively known to be occluded by previously drawn pixels. The rejection rate is up to 256 pixels per clock. Pixels that survive zcull then go to a fine-rasterization stage that generates detailed coverage information and depth values.

The depth test and update can be performed ahead of the fragment shader, or after, depending on current state. The SMC assembles surviving pixels into warps to be processed by an SM running the current pixel shader. The SMC then sends surviving pixel and associated data to the ROP.

Raster Operations Processor (ROP) and Memory System

Each ROP is paired with a specific memory partition. For each pixel fragment emitted by a pixel shader program, ROPs perform depth and stencil testing and updates, and in parallel, color blending and updates. Lossless color compression (up to 8:1) and depth compression (up to 8:1) are used to reduce DRAM bandwidth. Each ROP has a peak rate of four pixels per clock and supports 16-bit floating-point and 32-bit floating-point HDR formats. ROPs support double-rate-depth processing when color writes are disabled.

Antialiasing support includes up to 16× multisampling and supersampling. The *coverage-sampling antialiasing* (CSAA) algorithm computes and stores Boolean coverage at up to 16 samples and compresses redundant color, depth, and stencil information into the memory footprint and a bandwidth of four or eight samples for improved performance.

The DRAM memory data bus width is 384 pins, arranged in six independent partitions of 64 pins each. Each partition supports double-data-rate DDR2 and graphics-oriented GDDR3 protocols at up to 1.0 GHz, yielding a bandwidth of about 16 GB/s per partition, or 96 GB/s.

The memory controllers support a wide range of DRAM clock rates, protocols, device densities, and data bus widths. Texture and load/store requests can occur between any TPC and any memory partition, so an interconnection network routes requests and responses.

Scalability

The Tesla unified architecture is designed for scalability. Varying the number of SMs, TPCs, ROPs, caches, and memory partitions provides the right balance for different performance and cost targets in GPU market segments. *Scalable link interconnect* (SLI) connects multiple GPUs, providing further scalability.

Performance

The GeForce 8800 Ultra clocks the SP thread processor cores and SFUs at 1.5 GHz, for a theoretical operation peak of 576 GFLOPS. The GeForce 8800 GTX has a 1.35 GHz processor clock and a corresponding peak of 518 GFLOPS.

The following three sections compare the performance of a GeForce 8800 GPU with a multicore CPU on three different applications—dense linear algebra, fast Fourier transforms, and sorting. The GPU programs and libraries are compiled CUDA C code. The CPU code uses the single-precision multithreaded Intel MKL 10.0 library to leverage SSE instructions and multiple cores.

Dense Linear Algebra Performance

Dense linear algebra computations are fundamental in many applications. [Volkov and Demmel \[2008\]](#) present GPU and CPU performance results for single-precision dense matrix-matrix multiplication (the SGEMM routine) and LU, QR, and Cholesky matrix factorizations. [Figure B.7.3](#) compares GFLOPS rates on SGEMM dense matrix-matrix multiplication for a GeForce 8800 GTX GPU with a quad-core CPU. [Figure B.7.4](#) compares GFLOPS rates on matrix factorization for a GPU with a quad-core CPU.

Because SGEMM matrix-matrix multiply and similar BLAS3 routines are the bulk of the work in matrix factorization, their performance sets an upper bound on factorization rate. As the matrix order increases beyond 200 to 400, the factorization

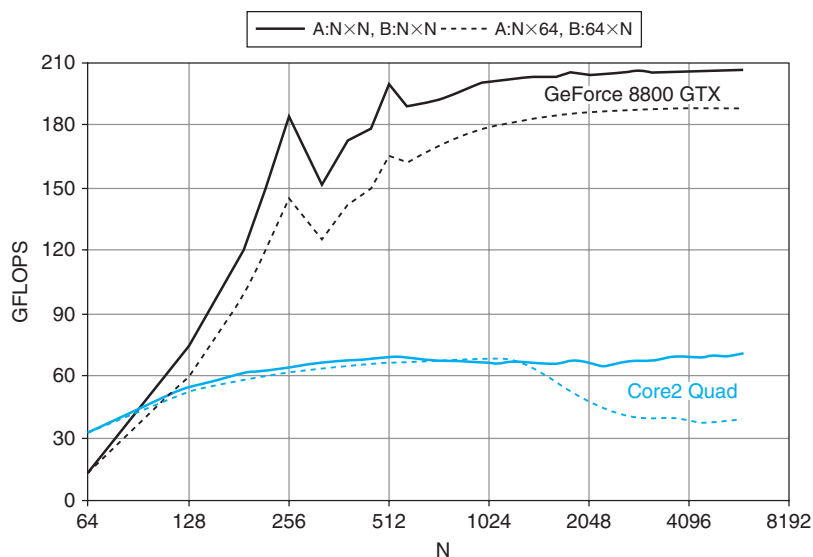


FIGURE B.7.3 SGEMM dense matrix-matrix multiplication performance rates. The graph shows single-precision GFLOPS rates achieved in multiplying square $N \times N$ matrices (solid lines) and thin $N \times 64$ and $64 \times N$ matrices (dashed lines). Adapted from Figure 6 of Volkov and Demmel [2008]. The black lines are a 1.35 GHz GeForce 8800 GTX using Volkov's SGEMM code (now in NVIDIA CUBLAS 2.0) on matrices in GPU memory. The blue lines are a quad-core 2.4 GHz Intel Core2 Quad Q6600, 64-bit Linux, Intel MKL 10.0 on matrices in CPU memory.

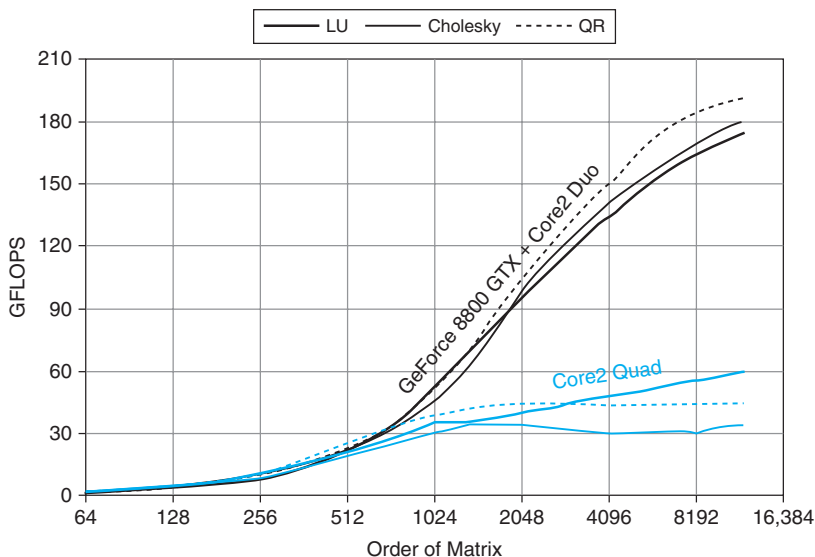


FIGURE B.7.4 Dense matrix factorization performance rates. The graph shows GFLOPS rates achieved in matrix factorizations using the GPU and using the CPU alone. Adapted from Figure 7 of Volkov and Demmel [2008]. The black lines are for a 1.35 GHz NVIDIA GeForce 8800 GTX, CUDA 1.1, Windows XP attached to a 2.67 GHz Intel Core2 Duo E6700 Windows XP, including all CPU-GPU data transfer times. The blue lines are for a quad-core 2.4 GHz Intel Core2 Quad Q6600, 64-bit Linux, Intel MKL 10.0.

problem becomes large enough that SGEMM can leverage the GPU parallelism and overcome the CPU–GPU system and copy overhead. Volkov’s SGEMM matrix-matrix multiply achieves 206 GFLOPS, about 60% of the GeForce 8800 GTX peak multiply-add rate, while the QR factorization reached 192 GFLOPS, about 4.3 times the quad-core CPU.

FFT Performance

Fast Fourier Transforms (FFTs) are used in many applications. Large transforms and multidimensional transforms are partitioned into batches of smaller 1D transforms.

Figure B.7.5 compares the in-place 1D complex single-precision FFT performance of a 1.35 GHz GeForce 8800 GTX (dating from late 2006) with a 2.8 GHz quad-Core Intel Xeon E5462 series (code named “Harpertown,” dating from late 2007). CPU performance was measured using the Intel *Math Kernel Library* (MKL) 10.0 FFT with four threads. GPU performance was measured using the NVIDIA CUFFT 2.1 library and batched 1D radix-16 decimation-in-frequency FFTs. Both CPU and GPU throughput performance was measured using batched FFTs; batch size was $2^{24}/n$, where n is the transform size. Thus, the workload for every transform size was 128 MB. To determine GFLOPS rate, the number of operations per transform was taken as $5n \log_2 n$.

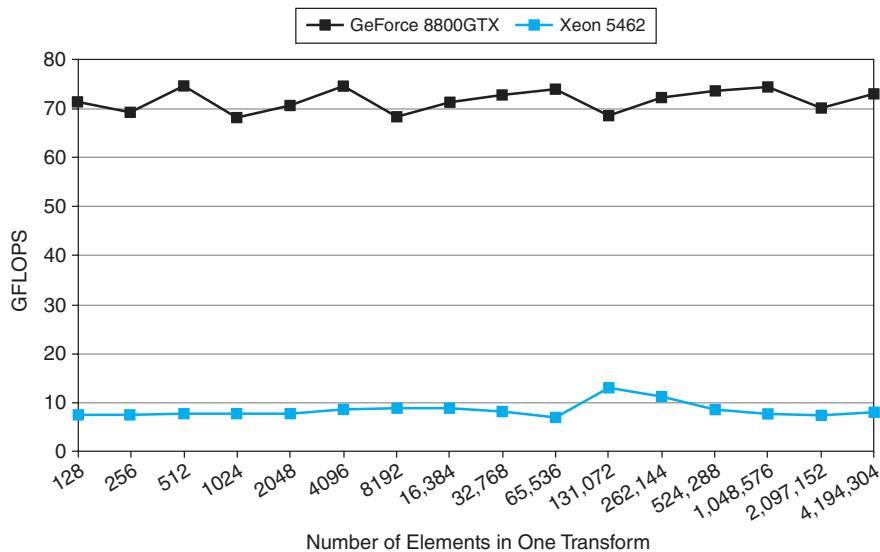


FIGURE B.7.5 Fast Fourier transform throughput performance. The graph compares the performance of batched one-dimensional in-place complex FFTs on a 1.35 GHz GeForce 8800 GTX with a quad-core 2.8 GHz Intel Xeon E5462 series (code named “Harpertown”), 6MB L2 Cache, 4GB Memory, 1600 FSB, Red Hat Linux, Intel MKL 10.0.

Sorting Performance

In contrast to the applications just discussed, sort requires far more substantial coordination among parallel threads, and parallel scaling is correspondingly harder to obtain. Nevertheless, a variety of well-known sorting algorithms can be efficiently parallelized to run well on the GPU. Satish et al. [2008] detail the design of sorting algorithms in CUDA, and the results they report for radix sort are summarized below.

Figure B.7.6 compares the parallel sorting performance of a GeForce 8800 Ultra with an 8-core Intel Clovertown system, both of which date to early 2007. The CPU cores are distributed between two physical sockets. Each socket contains a multichip module with twin Core2 chips, and each chip has a 4MB L2 cache. All sorting routines were designed to sort key-value pairs where both keys and values are 32-bit integers. The primary algorithm being studied is radix sort, although the quicksort-based `parallel_sort()` procedure provided by Intel's Threading Building Blocks is also included for comparison. Of the two CPU-based radix sort codes, one was implemented using only the scalar instruction set and the other utilizes carefully hand-tuned assembly language routines that take advantage of the SSE2 SIMD vector instructions.

The graph itself shows the achieved sorting rate—defined as the number of elements sorted divided by the time to sort—for a range of sequence sizes. It is

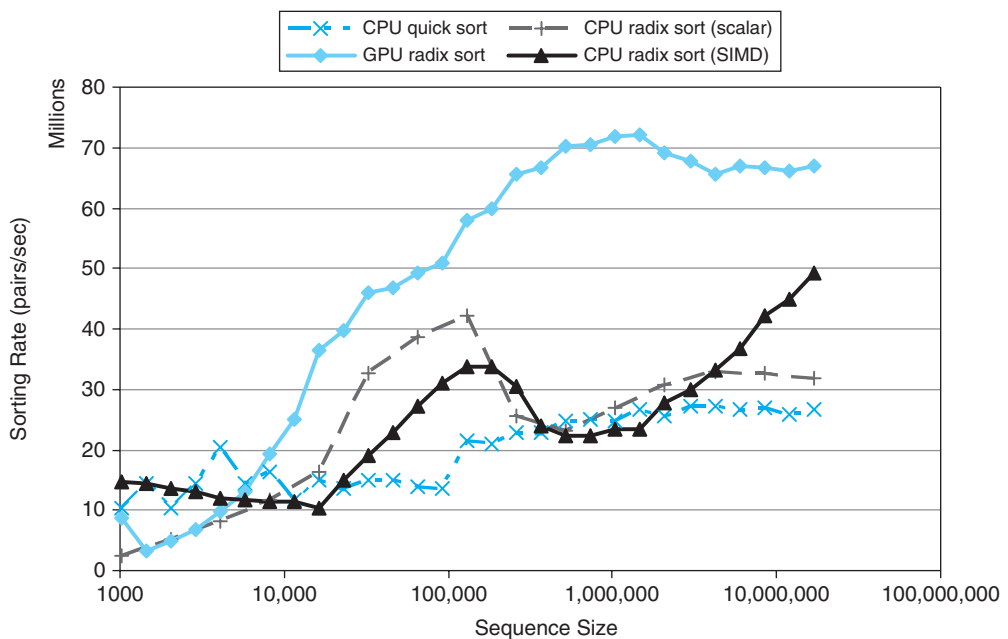


FIGURE B.7.6 Parallel sorting performance. This graph compares sorting rates for parallel radix sort implementations on a 1.5 GHz GeForce 8800 Ultra and an 8-core 2.33 GHz Intel Core2 Xeon E5345 system.